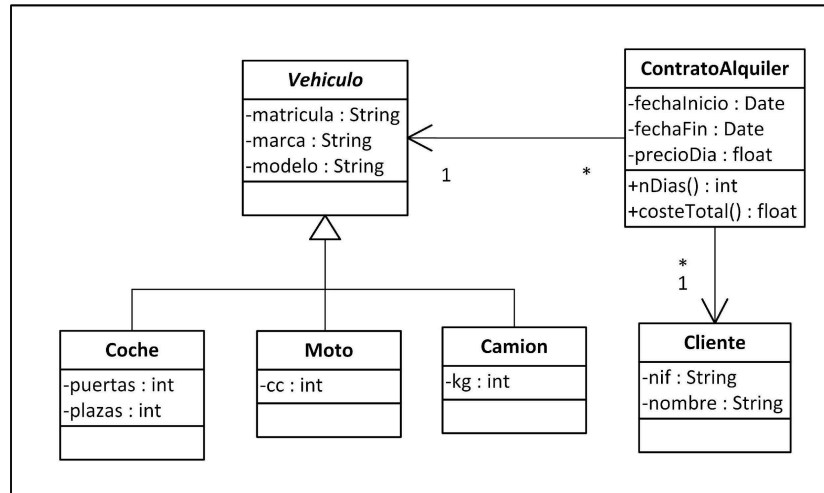


AA1. Diagrama de clases, Herencia y Java

Índice

Contenido

Preguntas:	3
1. ¿En qué casos declararemos una clase como abstracta?	3
2. ¿Qué es una relación de herencia en un diagrama de clases?	3
3. ¿En qué casos se aplica?	3
4. Indica las clases implicadas en la relación de herencia que hay en el diagrama de clases. ¿Cuál es la clase padre o superclase y cuáles son las clases hijas o subclases?	4
5. ¿Qué sentido tiene esta relación de herencia en este diagrama de clases?	4
Desarrollo	5
Autoevaluación	9



1. ¿En qué casos declararemos una clase como abstracta?

Utilizamos una clase como abstracta cuando queramos definir una estructura base o "plantilla" que contenga atributos y métodos comunes que luego queramos utilizar para otras clases, pero no necesitamos ni queremos instanciar objetos directos de ella.

En su lugar, el objetivo es instanciar únicamente las clases hijas (subclases) que hereden (extienden) de esta clase abstracta.

2. ¿Qué es una relación de herencia en un diagrama de clases?

Es una relación jerárquica, donde una clase derivada (subclase o clase hija) hereda los atributos y métodos de una clase base (superclase o clase padre).

En un diagrama UML, se representa mediante una línea continua con una flecha triangular vacía que apunta desde la subclase hacia la superclase. (en este caso Coche, Moto y Camión)

3. ¿En qué casos se aplica?

Se aplica cuando detectamos que dos o más clases comparten características (atributos) y comportamientos (métodos) idénticos.

En lugar de duplicar el mismo código en cada clase, nos aprovechamos de la herencia para abstraer y encapsular esos elementos comunes a una clase superior para que las subclases los hereden, facilitando la reutilización, la escalabilidad y el mantenimiento del código.

4. Indica las clases implicadas en la relación de herencia que hay en el diagrama de clases. ¿Cuál es la clase padre o superclase y cuáles son las clases hijas o subclases?

La superclase es Vehículo

Las clases extendidas (o subclases) son Coche, Moto y Camión.

Estas subclases heredan las propiedades matrícula, marca y modelo.

5. ¿Qué sentido tiene esta relación de herencia en este diagrama de clases?

Dado que un coche, una moto y un camión son vehículos tienen propiedades en común.

Por lo tanto tiene sentido agrupar los atributos que todos comparten (matrícula, marca y modelo) en la superclase Vehículo.

De este modo, se garantiza que todas las subclases posean estos datos comunes de forma automática sin reescribir código, al mismo tiempo que se permite a cada subclase definir sus propios atributos específicos (las puertas y plazas exclusivas del coche, los cc de la moto y los kg del camión).

Desarrollo

Lo primero es crear la clase abstracta de vehículo.

Declaramos los atributos que queremos extender luego al resto de clases (matrícula, marca y modelo) al igual que los métodos o atributos utilizaremos en los hijos.

```
You, 55 minutes ago | 1 author (You)
public abstract class Vehiculo {

    private String matricula;
    private String marca;
    private String modelo;

    public Vehiculo(String matricula, String marca, String modelo) {
        this.matricula = matricula;
        this.marca = marca;
        this.modelo = modelo;
    }

    public Vehiculo() {
    }

    public String getMatricula() {
        return matricula;
    }

    public void setMatricula(String matricula) {
        this.matricula = matricula;
    }

    public String getMarca() {
        return marca;
    }

    You, 2 days ago • feat: añade clase abstracta Vehiculo ...
    public void setMarca(String marca) {
        this.marca = marca;
    }

    public String getModelo() {
        return modelo;
    }

    public void setModelo(String modelo) {
        this.modelo = modelo;
    }

    @Override
    public String toString() {
        return "matricula='" + matricula + "', marca='" + marca + "', modelo='" + modelo + "'";
    }
}
```

Después crearemos las subclases, lo que hacemos es usar extends para aplicar la herencia y luego en el constructor usamos super() para traernos las propiedades que vamos a heredar

```

1  public class Coche extends Vehiculo {
2
3      private int puertas;
4      private int plazas;
5
6      public Coche(String matricula, String marca, String modelo, int puertas, int plazas) {
7          super(matricula, marca, modelo);
8          this.puertas = puertas;
9          this.plazas = plazas;
10     }
11
12     public Coche() {
13         super();
14     }
15     You, 1 hour ago • feat: añade constructor por defecto en Coche, M.
16
17     public int getPuertas() {
18         return puertas;
19     }
20
21     public void setPuertas(int puertas) {
22         this.puertas = puertas;
23     }
24
25     public int getPlazas() {
26         return plazas;
27     }
28
29     public void setPlazas(int plazas) {
30         this.plazas = plazas;
31     }
32
33     @Override
34     public String toString() {
35         return "Coche{" + super.toString() + ", puertas=" + puertas + ", plazas=" + plazas + "}";
36     }
37

```

```

You, 56 minutes ago | 1 author (You)
public class Camion extends Vehiculo {

    private int kg;

    public Camion(String matricula, String marca, String modelo, int kg) {
        super(matricula, marca, modelo);
        this.kg = kg;
    }

    public Camion() {
        super();
    }

    public int getKg() {
        return kg;
    }

    public void setKg(int kg) {
        this.kg = kg;
    }

    @Override
    public String toString() {
        return "Camion{" + super.toString() + ", kg=" + kg + "}";
    }
}
You, 2 days ago • feat: añade clase abstracta Vehiculo

```

```

100, 57 minutes ago | 1 author (You)
public class Moto extends Vehiculo {

    private int cc;

    public Moto(String matricula, String marca, String modelo, int cc) {
        super(matricula, marca, modelo);
        this.cc = cc;
    }

    public Moto() {
        super();
    }

    public int getCc() {
        return cc;
    }

    public void setCc(int cc) {
        this.cc = cc;
    }

    @Override
    public String toString() {
        return "Moto{" + super.toString() + ", cc=" + cc + "}";
    }
}
You, 2 days ago • feat: añade clase abstracta Vehiculo

```

Luego necesitamos las clases de Contrato, ya que las necesitamos para poder establecer las relaciones entre los objetos vehiculo y cliente:

```

1  import java.util.Date;
2
3  You, 1 hour ago | 1 author (You)
4  public class ContratoAlquiler {
5
6      private Date fechaInicio;
7      private Date fechaFin;
8      private float precioDia;
9      private Vehiculo vehiculo;
10     private Cliente cliente;
11
12     public ContratoAlquiler(Date fechaInicio, Date fechaFin, float precioDia, Vehiculo vehiculo, Cliente cliente) {
13         this.fechaInicio = fechaInicio;
14         this.fechaFin = fechaFin;
15         this.precioDia = precioDia;
16         this.vehiculo = vehiculo;
17         this.cliente = cliente;
18     }
19
20     public ContratoAlquiler() {
21     }
22
23     public Date getFechaInicio() {
24         return fechaInicio;
25     }
26
27     public void setFechaInicio(Date fechaInicio) {
28         this.fechaInicio = fechaInicio;
29     }
30
31     public Date getFechaFin() {
32         return fechaFin;
33     }
34
35     public void setFechaFin(Date fechaFin) {
36         this.fechaFin = fechaFin;
37     }
38
39     public float getPrecioDia() {
40         return precioDia;
41     }
42
43     public void setPrecioDia(float precioDia) {
44         this.precioDia = precioDia;
45     }
46
47     public Vehiculo getVehiculo() {
48         return vehiculo;
49     }
50
51 }

```

y también la clase Cliente

```

1  You, 1 hour ago | 1 author (You)
2  public class Cliente {
3
4      private String nif;
5      private String nombre;
6
7      public Cliente(String nif, String nombre) {
8          this.nif = nif;
9          this.nombre = nombre;
10     }
11
12     public Cliente(String nif) {
13         this.nif = nif;
14         this.nombre = "Sin especificar";
15     }
16
17     public Cliente() {
18     }
19
20     public String getNif() {
21         return nif;
22     }
23
24     public void setNif(String nif) {
25         this.nif = nif;
26     }
27
28     public String getNombre() {
29         return nombre;
30     }
31
32     public void setNombre(String nombre) {
33         this.nombre = nombre;
34     }
35
36     @Override
37     public String toString() {
38         return "Cliente{nif='" + nif + "', nombre='" + nombre + "'}";
39     }
40 }

```

Todas las clases tienen sus getters y setters correspondientes para setear o recuperar sus propiedades

Por último programamos la función inicio(), la cual contendrá la lógica del programa.

método el cual llamaremos en el método main()

```

You, 23 minutes ago | 1 author (You)
public class AA1_AlquilerVehiculos {

    Run | Debug
    public static void main(String[] args) {
        AA1_AlquilerVehiculos aal = new AA1_AlquilerVehiculos();
        aal.inicio();
    }

    void inicio() {
        Coche coche = new Coche(matricula: "123HABC", marca: "Seat", modelo: "Ibiza", puertas: 5, plazas: 5);
        Moto moto = new Moto(matricula: "5678DEF", marca: "Honda", modelo: "CBR500R", cc: 500);
        Camion camion = new Camion(matricula: "9012GHI", marca: "Volvo", modelo: "FH16", kg: 20000);

        Cliente cliente1 = new Cliente(nif: "12345678A", nombre: "Laura Garcia");
        Cliente cliente2 = new Cliente(nif: "87654321B", nombre: "Marc Puig");

        Date inicial = crearFecha(year: 2026, month: 3, day: 1);
        Date fin1 = crearFecha(year: 2026, month: 3, day: 8);
        ContratoAlquiler contrato1 = new ContratoAlquiler(inicial, fin1, precioDia: 45.0€, coche, cliente1);

        Date inicio2 = crearFecha(year: 2026, month: 3, day: 5);
        Date fin2 = crearFecha(year: 2026, month: 3, day: 10);
        ContratoAlquiler contrato2 = new ContratoAlquiler(inicio2, fin2, precioDia: 30.0€, moto, cliente2);

        Date inicio3 = crearFecha(year: 2026, month: 3, day: 3);
        Date fin3 = crearFecha(year: 2026, month: 3, day: 9);
        ContratoAlquiler contrato3 = new ContratoAlquiler(inicio3, fin3, precioDia: 120.0€, camion, cliente1);

        System.out.println("=== Vehiculos ===");
        System.out.println(coche);
        System.out.println(moto);
        System.out.println(camion);

        System.out.println("=== Clientes ===");
        System.out.println(cliente1);
        System.out.println(cliente2);

        System.out.println("=== Contratos de alquiler ===");
        System.out.println(contrato1);
        System.out.println(contrato2);
        System.out.println(contrato3);
    }

    private Date crearFecha(int year, int month, int day) {
        Calendar cal = Calendar.getInstance();
        cal.set(year, month - 1, day, 0, 0, 0);
        cal.set(Calendar.MILLISECOND, 0);
        return cal.getTime();
    }
}

```

Una vez construida la lógica, lo que queda es compilar el .jar y ejecutarlo:

```

doplax:AA1/ (main) $ javac -d out/production/AA1_AlquilerVehiculos src/*.java [20:36:44]
doplax:AA1/ (main) $ echo "Main-Class: AA1_AlquilerVehiculos" > manifest.txt [20:38:33]
doplax:AA1/ (main) $ jar cfm AA1_AlquilerVehiculos.jar manifest.txt -C out/production/AA1_AlquilerVehiculos . [20:38:46]
doplax:AA1/ (main) $ rm manifest.txt [20:38:50]
doplax:AA1/ (main) $ java -jar AA1_AlquilerVehiculos.jar [20:38:54]

=== Vehiculos ===
- Coche: Seat Ibiza (1234ABC) | Puertas: 5 | Plazas: 5
- Moto: Honda CBR500R (5678DEF) | Cilindrada: 500 cc
- Camión: Volvo FH16 (9012GHI) | Capacidad: 20000 kg
=== Clientes ===
- Cliente: Laura Garcia | NIF: 12345678A
- Cliente: Marc Puig | NIF: 87654321B
=== Contratos de alquiler ===
- Contrato -> Cliente: Laura Garcia (12345678A)
  Vehículo: Seat Ibiza [1234ABC]
  Fechas: 01/03/2026 a 08/03/2026
  Resumen: 7 días x 45.0€/día = Coste Total: 315.0€
- Contrato -> Cliente: Marc Puig (87654321B)
  Vehículo: Honda CBR500R [5678DEF]
  Fechas: 05/03/2026 a 10/03/2026
  Resumen: 5 días x 30.0€/día = Coste Total: 150.0€
- Contrato -> Cliente: Laura Garcia (12345678A)
  Vehículo: Volvo FH16 [9012GHI]
  Fechas: 03/03/2026 a 09/03/2026
  Resumen: 6 días x 120.0€/día = Coste Total: 720.0€
doplax:AA1/ (main) $ | [20:38:59]

```

Video

<https://youtu.be/GuOpOh-gVWA>

Autoevaluación

(antes de dar mi autoevaluación quería comentar que actualmente tengo 5 años en el sector, adjunto mi [GitHub](#))

En esta práctica he podido utilizar Java para repasar y aplicar los fundamentos más académicos de la Programación Orientada a Objetos. Aunque por mi experiencia laboral ya conocía bien de qué iba el tema y cómo estructurar clases, este ejercicio me ha resultado muy útil para volver a las bases: traducir estrictamente un diagrama UML a código, gestionar correctamente la herencia con clases abstractas, aplicar la sobrecarga de constructores clásica de Java y refrescar el uso de clases más antiguas como Date y Calendar para la lógica de fechas.

Repasando la rúbrica de la asignatura, considero que el proyecto cumple con todos (o la mayoría) de los requisitos.